Small Dragon Hatchling Models: Alphabet, Memory, and Fine-tuning

Dev Mehra
devxmehra@gmail.com
October 2026

This work is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Code, models, and transcripts: github.com/d3vmeh/small-dragon-hatchling
Models: huggingface.co/collections/d3vmeh/small-dragon-hatchling-models

Summary

I trained a set of Dragon Hatchling models of sizes 25M, 50M, and 100M parameters on two alphabets, bytes and Byte Pair Encoding (BPE) tokens, using the same text at 25M and 50M and the same amount and mix of text at 100M. BPE outperforms bytes by 4 to 8% in bits per character on the same held-out text. The two alphabets score within 0.02 bpc of each other when given roughly the same amount of context, which suggests that part of the gap comes from each BPE token position seeing four times more characters. Compared to a rotary transformer of matched size trained on the same amount and mix of text, BDH scores about 5% better in bits per character on bytes, at about six times the compute per position. On BPE, the two are within 2.4% of each other at every size, so the byte-level advantage mostly disappears.

I built streaming inference and found that feeding each position once into the synaptic state ρ matches recomputing all the previous context, and with a sliding context window the model predicts as well as a fresh model given the last 480 positions, through 4,096 positions of text, which is past the 512-position block it was trained on. I tested recall by planting a fact in each model's context. In the BDH base models, the planted sentence makes the word hundreds to tens of thousands of times more likely at distances up to 128 characters. The BDH byte base models mostly fail to recall the correct word while the transformers do. However, chat tuning the models teaches them to do so, raising byte recall from about 17% to 93% at 128 characters. For the byte model only the v3 data produced this, whereas both the v2 and v3 tuning sets worked for BPE. Compared to v2, v3 added list items and changed the story questions at the same time, so I did not determine which change produced this. At 100M, the distance the planted fact can be recalled from is several times larger with BPE than bytes after tuning.

LLM judges comparing single replies cannot separate the v2 or v3 models or the two alphabets, but a multi-turn exam can. When asked questions about a story it has just written, the tuned BPE model gives answers supported by that story 36% of the time, and the tuned byte model 16% of the time.

By varying the number of layer passes at inference, I found that BDH models need to run inference at the same number of layers they were trained on, otherwise their output quality drops rapidly. Looking at the neuron density of the 25M, 50M, and 100M models, the larger models fire a smaller proportion of their neurons.

Compute and budget were limited, so most measurements come from a single training run per model. The code, result files, and transcripts, as well as the six base models and the two v3 chat models are public (links on the title page).

Section 1: Models and Tools

I trained six models on the Dragon Hatchling (BDH) architecture described in the 2025 paper "The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain" [1]. A BDH model contains six layers that share one set of weights, with each layer having four attention heads and a layer of sparsely activated neurons. With D as the size of the vector embedding for each byte/token, there are $N = 32 \cdot D$ neurons per head. In addition to the six models shown in the table below, I also trained a second seed of the 25M and 50M pairs, a four-pass 25M, and six transformer models as controls. I also created five chat-tuned versions of the 100M models, described in the sections that use them. I trained all the models on a context block of up to 512 positions using data from the FineWeb-Edu [2] and TinyStories [3] datasets.

I paid about $90 for a cloud H100 to pretrain the 100M byte model and run its first two chat fine-tunes, and about $15 more to train the 100M byte transformer. I trained and ran the rest of the models locally on the integrated Radeon 8060S GPU of an AMD Ryzen AI Max+ 395 (Strix Halo) and an NVIDIA RTX 4090. The tuning dataset and judging API calls cost about $60.

Model	Parameters	D	Alphabet	Training text	Hardware, time	Chat versions
byte-100M	100.9M	512	bytes	5.4 GB, 83% FineWeb-Edu + 17% TinyStories	H100, 27 h	v1, v2, v3
BPE-100M	100.9M + vocab	512	BPE-8k	5.4 GB, same mix, tokenized	RTX 4090, 20 h	v2, v3
byte-50M	56.8M	384	bytes	3.07 GB of the same mix (54 bytes/param)	RTX 4090, 25 h	none
BPE-50M	56.8M + vocab	384	BPE-8k	the same 3.07 GB, tokenized	RTX 4090, 7 h	none
byte-25M	25.3M	256	bytes	1.37 GB of the same mix (54 bytes/param)	RTX 4090, 7 h	none
BPE-25M	25.3M + vocab	256	BPE-8k	the same 1.37 GB, tokenized	RTX 4090, 2 h	none

The six base models trained for this paper. Times are wall-clock training times. The BPE training takes fewer steps on the same text which is why it finishes sooner. The 100M models were trained on the same amount of text with the same 83% to 17% proportion of datasets, but not the exact same documents. The 25M and 50M pairs share their text exactly.

Section 2: Alphabet

I trained the models across two different alphabets: bytes and tokens. For byte-level prediction, the model learned to predict byte by byte [4, 5]. For token-level prediction, I used Byte Pair Encoding (BPE) [6] with 8,192 distinct tokens, trained with the Hugging Face tokenizers library [7]. It is byte-level BPE with GPT-2 style pre-tokenization, no normalization, and all 256 byte values in the base alphabet, so every byte string tokenizes and decodes back exactly and the two alphabets score the same characters. For each size, I trained one byte model and one BPE model on the same text, except at 100M where they share the same amount and mix of text but not the same exact documents. The training text is made up of a combination of 83% FineWeb-Edu text and 17% text from TinyStories.

The model size is set by the width D of the residual stream, the length of the vector that represents each byte or token. I refer to a single byte or token as an atom. The three model sizes use D = 256, 384, and 512, resulting in 25.3M, 56.8M, and 100.9M parameters (labeled as 25M, 50M, and 100M below). The six layers all share the same set of weights, so the parameters roughly scale as $384 \cdot D^2$. Each model was given 54 bytes of training text per shared-layer parameter.

Each model is scored by cross-entropy on 3 MB of held-out text that includes part of FineWeb-Edu outside of both training datasets, as well as TinyStories validation split, fed one byte or token at a time. The surprisal is $-\log_2$ of the probability the model assigned to the correct next symbol. Summing the surprisal and dividing by the number of characters gives bits per

character (bpc). Dividing by characters instead of predictions is necessary to compare the alphabets, because the BPE model makes fewer predictions over the same size text.

An earlier matched-compute comparison at 57M showed a 12.6% gap. But, at matched compute, the BPE model read about four times more text, so the comparison below holds the text fixed instead (Appendix A5).

Size (D)	Params	Bytes bpc	BPE bpc	Gap
25M (256)	25.3M	1.178	1.126	0.052 (4.4%)
50M (384)	56.8M	1.104	1.019	0.084 (7.6%)
100M (512)	100.9M	1.042	0.964	0.078 (7.5%)

Table 1: Bits per character on 3 MB of held-out text (part of FineWeb-Edu outside both training sets, as well as TinyStories validation split), scored in 512-position blocks. Lower is better. Gap is bytes minus BPE as a fraction of the byte score. One training seed per size. Both held-out parts are shown separately in Appendix A1. Parameter counts are the byte models'. The BPE models have the same number of shared-layer weights in addition to larger embedding and output tables, which make them 8 to 16% larger in total.

The gap widens from 25M to 50M, then holds at 100M. Part of the gap comes from the context span (Table 2). I scored each position with its context limited to between k and 2k positions, for k from 8 to 256.

Size	Bytes, 256-512 chars behind	BPE, 256-512 chars behind (64-128 tokens)	BPE, 1,000-2,000 chars behind (256-512 tokens)
25M	1.039	1.044	1.026
50M	0.962	0.946	0.922
100M	0.902	0.895	0.868

Table 2: Bits per character on 1.5 MB of the same held-out text, only counting positions whose context length falls in that stated band. The held-out text has about 3.9 characters per token, so 64 to 128 tokens is about 250 to 500 characters. All six bands for both alphabets are in Appendix A2. Lower is better. One training seed per size.

Table 2 shows that BPE positions with 64 to 128 preceding tokens (about 256 to 512 characters) score within 0.02 bpc of byte positions with 256 to 512 preceding characters at every model size. So about 0.02 to 0.03 bpc of BPE's advantage, roughly a third of the Table 1 gap at each size, is consistent with seeing roughly four times as many preceding characters, rather than better prediction on the same text. The bands match the amount of context each model has, not the exact positions being scored, and the band text is a 1.5 MB subset with a different mix of the two sources than Table 1, so the third is a rough share rather than an exact split of the gap. A

byte-level simulation study reaches a related conclusion, attributing most of the subword advantage to training throughput and subword-boundary priors [8].

Table 1 scores each 512-position block from its first position, so every block includes early slots. At the tenth position of a block, a byte model has only 10 characters to predict from, about a word-and-a-half, while a BPE model has 10 tokens or around 40 characters. Comparing the bytes column with the BPE 1,000 to 2,000 characters column in Table 2, where both models have hundreds of preceding characters, the 100M gap is approximately 3.7%. On the same 1.5 MB text, the block-averaged gap is 8.1% (Appendix A2), so at 100M the gap with long context is less than half the block-averaged gap. The two numbers average over different positions, so this is only indirect evidence that the early slots carry much of the gap.

Extrapolating from the 50M and 100M results, I predict a block-averaged gap of 7 to 8% at 500M.

Section 3: Streaming Inference over Synaptic State

BDH's attention keeps a fixed-size state per layer and head in the form of an NxD matrix, which the BDH paper calls ρ, a compressed form of the neuron-to-neuron matrix σ, and describes as fast weights updated by a Hebbian rule [1, 9]. At each new position, the rows corresponding to each neuron activated by the symbol are added into a weighted sum (weighted by how strong each sparse activation was), which becomes that position's attention output as a vector. Then, its residual vector at that layer gets added to each row whose corresponding neuron is active, scaled by that neuron's activation.

Because the state is fixed-size, a new position only needs to look at ρ, not the earlier text [10]. So instead of rerunning the entire context for every new position, the text can be fed one position at a time, just keeping all 24 copies of ρ (6 layers x 4 heads per layer).

On an earlier 57M byte model I tested running on the Strix Halo integrated GPU, one new position costs 45 ms at any context length, while recompute costs grow as context grows, 81 ms at 128 positions and 7.9 s at 4,096 (Appendix A3). To check that streaming is accurate, I compared streaming and full-recompute logits at every position of a 512-position block for the 57M byte, 100M byte, and 100M BPE models. The largest difference was 2e-5 and, with both methods always taking the top choice, the next 40 symbols were identical (Appendix A3b).

100M model	0-512	512-1024	1024-1536	1536-2048	2048-2560	2560-3072	3072-3584	3584-4096
bytes, fresh block	1.100	1.105	1.009	1.139	1.172	1.215	1.357	1.153
bytes, recompute last 480	1.100	1.019	0.920	1.098	1.069	1.135	1.248	1.136
bytes, no window	1.100	3.667	3.711	3.960	3.907	4.155	4.184	4.137
bytes, window 480	1.098	1.020	0.917	1.101	1.086	1.141	1.239	1.141
BPE, fresh block	0.997	1.179	1.252	1.173	1.158	1.132	1.318	1.343
BPE, recompute last 480	0.997	1.143	1.212	1.121	1.127	1.106	1.270	1.196
BPE, no window	0.997	1.677	1.786	1.759	1.715	1.757	2.052	2.209
BPE, window 480	0.997	1.144	1.215	1.123	1.134	1.115	1.291	1.243

Table 3: bpc per 512-position chunk in the first of ten passages, which starts at the beginning of the held-out file, while streaming 4,096 positions of held-out text through the two 100M models. “fresh block” is the model with context restarted at each chunk. “no window” is streaming without clearing ρ. “window 480” is streaming with a sliding 480-position context window. Lower is better. One training seed. Chunks are counted in positions, so a BPE chunk spans about four times the amount of text. Cost per position on the Strix Halo where I ran inference was 91 ms with the window and 439 ms for recompute for the byte model (90 ms and 456 ms for BPE).

Beyond the trained context window, the output degrades rapidly, for two reasons that I did not directly separate in this test: the model is reading at position offsets it never saw in training and ρ has filled up with everything ever written. A sliding window that subtracts a position’s contributions from every ρ once it falls 480 positions behind removes the degradation of the output in practice. To make a fair benchmark, I scored a model that reread all of the last 480 positions from scratch at each step, which is exact but about five times slower. Streaming ten passages of 4,096 positions of held-out text, the model with the window stays within 0.01 bpc of the rereading model on average, and within 0.05 in every chunk of every passage. Without the window, the byte model’s score past 512 positions is 2.7 to 4.9 times worse than if the plain model was restarted at each chunk, and the BPE model is 1.4 times to 2.5 times worse, as ρ gets filled (Appendix A4). While this subtraction occurs on every layer’s ρ, it is only exact at the first layer. This is because later positions would have already read the removed position before writing themselves, leaving traces of that position. Similarly, a cached sliding window in a transformer is not equivalent to recomputing over the window [11].

Section 4: Memory

To test whether a model can retrieve a fact planted earlier in its context, it is given text beginning with something like: “The secret word is banana.”, followed by filler story text (a Shakespeare

passage) whose length is the distance, and ending with “The secret word is”. At that point, the model is scored on the probability it assigns to the true word followed by its period, as in Section 2. I ran 30 trials at each distance with 10 different secret words, each planted in three different filler texts, and measured the models’ gain and recall. Gain is the surprisal of the word in the same filler with the planting sentence removed, minus its surprisal with the sentence present, averaged over the 30 trials. Recall is the share of the 30 trials where the model’s top choice was correct at every symbol of the word and its final period.

Distance (chars)	100M byte base gain (nats)	100M byte base recall	Bytes v3 gain	Bytes v3 recall	BPE v3 gain	BPE v3 recall
0	10.2	100%	11.4	90%	12.0	100%
64	11.1	57%	13.9	67%	13.1	97%
128	9.7	17%	14.3	93%	12.0	93%
256	8.7	0%	12.6	13%	12.8	93%
384	8.8	0%	13.5	7%	12.0	67%
600	1.2	0%	0.2	0%	12.5	90%
1,024	1.3	0%	2.5	0%	10.2	33%

Table 4: Gain (nats of surprisal removed by the planted phrase, averaged over 30 trials), and recall (share of the 30 trials where the top choice was the planted word), by distance in characters. The trained block is 512 positions, which is 512 characters for bytes and about 2,000 for BPE (fewer on the Shakespeare filler text, see Table 5b). For the byte models, the 600 and 1,024 rows are beyond the 512-character block. Streamed with no window, so the 600 and 1,024 rows read past the block through ρ. One phrasing of plant and cue. Ten recorded words (banana, purple, rocket, pencil, tiger, window, silver, castle, honey, lantern) in three filler texts, 30 trials per cell. At distance 0 the filler is empty, so that row is ten distinct prompts counted three times.

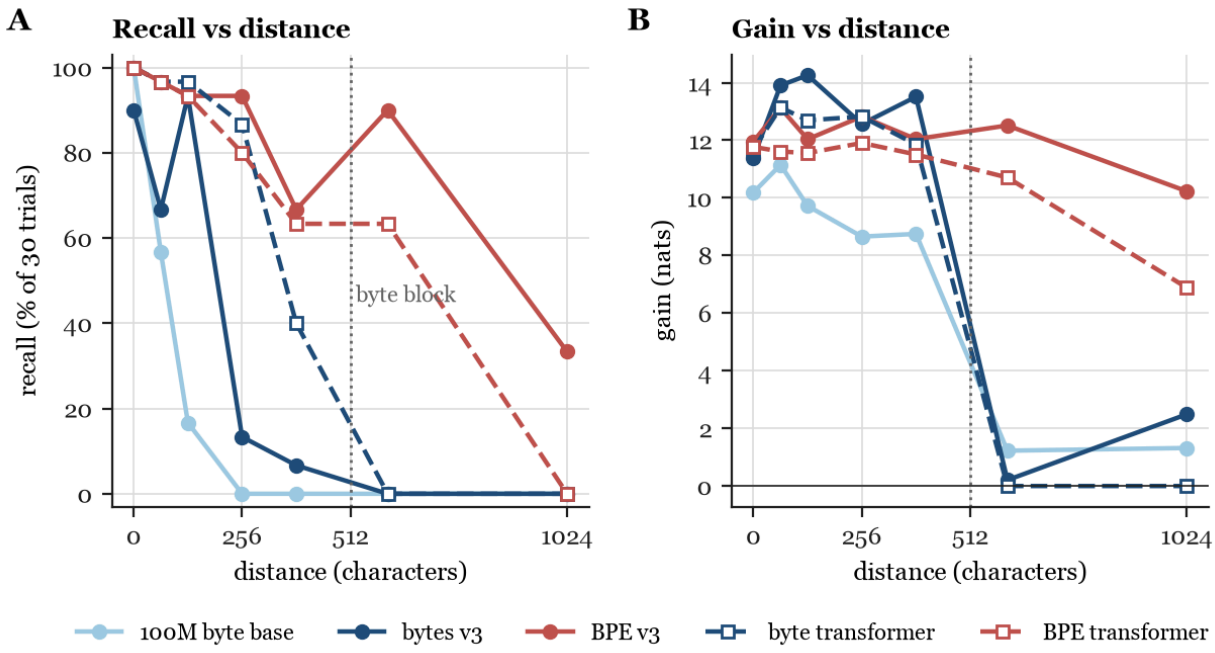


Figure 1: Planted-fact recall (A) and gain (B) against distance for the 100M models. Solid lines are the BDH models of Table 4 (byte base, bytes v3, BPE v3), dashed lines are the untuned 100M transformers of Tables 5a and 5b. 30 trials per point, ten recorded words in three filler texts. BDH is scored with no window, the transformers are scored with their 512-position block. The dotted line marks the 512-character training block of the byte models. The BPE block is 512 tokens, about 1,700 characters on this filler (past the right edge of the plot).

Reach is the largest distance at which recall is at least 50%.

The 100M base received no tuning and its gain remains about 9 to 11 nats (natural log units of surprisal, 1 nat = 1.44 bits) at every distance up to 384 characters, while its recall falls to 0% by 256 (Table 4, Figure 1). Planting the word under a different key, such as “The dog’s name is”, while still asking for the secret word gives similar gain (Appendix A23), so the gain is not bound to the question wording. For the byte bases, 6 to 8 nats come from the letters after the first, scored with the true preceding letters in place, so the planting mostly helps the model continue the word rather than begin it (Appendix A14). At 128 characters onwards, the planted phrase raises the first letter by only 0.4 to 1.4 nats, which is usually not enough to make it the top choice. Tuning increases the first-letter gain to 2.8 nats at 128 characters for bytes v3 and recall improves. The byte transformers hold the fact at 9 to 13 nats at every distance inside their 512-character training block (Appendix A19), while the BPE transformers decay from about 9 to 12 nats down to 6 as the distance grows to 1,024 characters (Appendix A20). For the byte models, the transformer holds it more at every distance inside the block by 1.4 to 4.4 nats. Past the 512-character block, the byte models’ gain drops sharply, to 2 to 3 nats at 50M, about 1 nat at 100M, and near zero at 25M.

Distance (chars)	Transformer 25M	BDH 25M base	Transformer 50M	BDH 50M base	Transformer 100M	BDH 100M base
0	80%	60%	100%	100%	100%	100%
64	90%	3%	100%	7%	97%	57%
128	60%	3%	100%	3%	97%	17%
256	0%	0%	30%	0%	87%	0%
384	0%	0%	0%	0%	40%	0%
600	0%	0%	0%	0%	0%	0%
1,024	0%	0%	0%	0%	0%	0%

Table 5a: Recall (percentage of the 30 trials where the greedily selected top choice was the planted word). Same runs as Appendix A19. 30 trials per entry, one standard deviation of a 30-trial rate is up to 9 points, treating the trials as independent. Recall also varies by filler text well beyond that: bytes v3 at 64 characters scored 1, 9 and 10 of 10 on the three fillers (probe result files are in the GitHub repository).

Distance (chars)	Transformer 25M	BDH BPE 25M base	Transformer 50M	BDH BPE 50M base	Transformer 100M	BDH BPE 100M base
0	30%	0%	100%	40%	100%	90%
64	73%	3%	73%	20%	97%	57%
128	63%	0%	77%	7%	93%	53%
256	10%	0%	47%	0%	80%	33%
384	10%	0%	23%	0%	63%	20%
600	13%	0%	7%	0%	63%	13%
1,024	0%	0%	0%	0%	0%	0%

Table 5b: Recall for BPE transformers and BPE BDH models, same runs as Appendix A20. 30 trials per entry, one standard deviation of a 30-trial rate is up to 9 points, treating the trials as independent. Distances are in characters. On this filler, 600 characters is 187 to 205 BPE positions and 1,024 is 302 to 324 across the three filler texts, so every distance is inside the 512-position block.

The 100M bytes and BPE models were both fine-tuned for 800 steps on the v3 dataset, which contains stories followed by questions with answers that are inside the story, as well as lists of items followed by position questions. For the byte model, recall at 128 characters rose from 17% for the base to 93% for the v3 fine-tuned model, then decayed to 13% at 256 while gain remained at 12.6 to 14.3 nats the entire time. The v3 data never contained the phrase “The secret word is”. When planting the word as the dog’s name while still asking for the secret word, bytes v3 drops from 28 of 30 to 11 of 30 at 128 characters (Appendix A23). The released v1 and v2 tunes did not raise byte recall above the untuned base model (Appendix A6), although the v2

data does when tuned for only 800 steps (Appendix A27). v1 was tuned only on stories and recalled less often than the base, and v2 raised gain to 18 nats but rarely picked the correct word.

At 25M and 50M both untuned byte and BPE transformers reach 128 characters, at 100M the byte transformer reaches 256 and the BPE transformer reaches 600, so for untuned transformers reach extends with the alphabet at 100M but not at smaller sizes (Table 5a, 5b). On the held-out text from Table 1, the byte transformers score about 5% higher bpc at every size, so their better recall is not mainly explained by them being better language models. However, tuning the BDH models brings the bytes v3 recall to 93% at 128 characters, matching the untuned transformers there, but not at 256 characters, where the 100M transformer reaches 87% and the bytes v3 model reaches only 13%, the same shape as the recall gap that separates attention from efficient attention-free models [12].

For bytes v3, most of the gain is on the letters after the first, while for BPE v3 the words are one to three tokens and about 70% of the gain is on the first token (Appendix A14). The BPE v3 model maintained gain and recall across a greater distance. With the word planted as the dog's name, BPE v3 still scores 29 of 30 at 128 characters and 20 of 30 at 600 (Appendix A23). This alone does not confirm whether the model retrieves the word based on its key or just names the unusual word in its context. To separate them, I planted two words under two keys in the same context, "The secret word is X" and "The dog's name is Y" then asked for the secret word. BPE v3 answered X 25 out of 30 times at 64 characters and 24 of 30 at 600, versus 4 of 30 for Y (Appendix A24), so with this question it picks the word under the key the question names.

At 1,024 characters, BPE v3 has 10.2 nats of gain with 33% recall. Bytes v3 loses recall between 128 and 256 characters, and BPE v3 between 600 and 1,024, which on this filler is 187 to 324 token positions. Those position ranges overlap so the difference in characters is consistent with token length rather than a difference in memory ability.

Section 5: Tuning

Tuning these models involves standard next-symbol training over a few thousand conversation-shaped documents. Loss is calculated only on the model's response. I tuned with three datasets, which I have labeled v1, v2, and v3. Only the 100M bytes and BPE bases are tuned here.

The v1 dataset was built from thousands of story requests, with the phrase "User: tell me a story about [topic]" followed by a TinyStories story as the assistant's reply. The topic was pulled from the story.

v2 used the same conversational shape as v1 but included 1,620 dialogues generated by Claude Opus 5 based on category prompts in the TinyStories vocabulary, and every dialogue was under 512 bytes so the byte model could hold the whole dialogue. The dialogues included four categories: 900 multi-turn conversations (a short story followed by two questions with answers in the story), and smaller sets of micro conversations (greetings and small tasks such as "count to five"), constraints such as "answer in one word", and stops to teach the model the end marker. The pool, which contained these dialogues repeated three times and 3,000 v1 story documents, had 7,860 examples total. Training ran 2,000 steps in batches of 32, about eight passes over the pool. However, the problem with the multi-turn documents was that the user's request often includes the details, so follow-up questions could be answered just from the request. This was a flaw in my generation prompt, where I had named the topic in the request. Additionally, training loss for v2 had dropped to 0.045 nats per byte, suggesting that the model was memorizing its training documents. Retuning with the v2 data for 800 steps and the v3 data for 2,000 steps showed that the held-out cost follows mainly the step count, with the v3 data costing 0.02 to 0.05 bpc more at matched steps, and the training loss dropped to around 0.02 to 0.05 nats per byte or per token with either dataset at 2,000 steps. At 800 steps, both datasets land within 0.05 bpc of each other on held-out text, and at 2,000 steps, both lose 57 to 61% on bytes (Appendix A26). Across the 120 prompts, blinded LLM judge panels showed no detectable preference between the two datasets at equal step counts. With the step count held constant, the byte model tuned on the v3 data recalls the planted fact at 128 characters more often than v2 (28 of 30 trials compared to 14 of 30 at 800 steps and 24 versus 5 at 2,000 steps). For BPE all four tunes recall similarly, 28 of 30 at 128 characters compared to 16 for the untuned model (Appendix A27).

The v3 data contained v2's micro, constraint, and stop documents, but replaced the multi-turn category with two others. The first was 900 grounded dialogues: a generic request such as "a story about a pet", a story that invents specific details (names, colors, places), and questions about those invented details, so the answers only exist in the story to avoid the shortcut in v2. The second was 360 ordinal dialogues, where the user asks a listing question like "name three animals", the assistant lists them, and the user asks for the first, second, last, or how many. The

1,980 dialogues were repeated twice and joined by the same 3,000 story documents, a pool of 6,960 examples. I reduced the steps from 2,000 to 800, about four passes, to prevent the memorization seen in v2.

Five chat-tuned models were produced: bytes v1, v2, and v3, and BPE v2 and v3. I compared them in five panels. Each panel gives two models the same 120 single-turn prompts consisting of stories, questions, explanations, and micro-tasks. All replies in the panels and in the multi-turn exam were sampled at a temperature of 0.8 with top-k 40, streamed with the 480-position window and stopped at the end marker or when the model started a user turn, with a cap of 420 bytes for the byte models and 130 tokens for BPE. The two answers were shown to a judge in random order with no model names. The judge rates coherence, instruction following, fluency, and an overall verdict, each as A, B, or tie. Table 6 counts the overall verdict. I used a panel of three LLM judges: Anthropic Claude Opus 5, OpenAI GPT 5.6 Sol, and DeepSeek V4 Pro. Each prompt's verdict was the two-of-three majority of the judges.

v1 and v2 came from the same base with the same 2,000 steps. In the v1-versus-v2 panel, v2 won 97 while v1 won 22. v1 only trained on the stories category, so most of the margin shows that a model trained on a task is able to answer it. On stories only, v2 beat v1 25 to 14 ($p = 0.11$ on an exact binomial split), so I cannot say that the new documents helped on the shared task (Appendix A12).

Panel	A	B	A wins	B wins	No majority	Margin	p (exact binomial)
curriculum, bytes	v1	v2	22	97	1	75	2e-12
	v2	v3	48	54	18	6	0.62
curriculum, BPE	v2	v3	51	47	22	4	0.76
alphabet at v2	bytes v2	BPE v2	47	57	16	10	0.38
alphabet at v3	bytes v3	BPE v3	54	48	18	6	0.62

Table 6: The five single-turn judge panels at 100M. 120 prompts per panel, consisting of 40 stories, 40 questions, 24 explanations, and 16 micro-tasks (grouped by keyword). Each judge saw each pair blindly and in random order (Opus returned no ruling on 9 pairs of the v1 vs v2 panel and 8 of the bytes v2 vs BPE v2 panel because of API errors, so those pairs had two judges). Prompt winner is decided by a two-of-three majority. No majority means no model was chosen by two judges, with ties not counting as a choice. Margin is the winner's lead among decided prompts (prompts with a majority verdict). p is calculated from an exact two-sided binomial test on the decided prompts assuming the two models are equally good (fair coin flip).

In the other four panels, the margins are 4 to 10 prompts, with p between 0.38 and 0.76, so no difference is detectable at this sample size. With about 100 decided prompts, a panel needs 61 wins before a result counts. Each pair of judges gives the same verdict (A, B, or tie) on 80 to

94% of the prompts both of them ruled on, in every panel, so the ties are not caused by judge disagreement. Of the 16 to 22 prompts per tied panel with no majority verdict, only 1 or 2 had two judges choosing opposite models. The rest were called a tie by all three judges or by two of them. To test the models' memory across turns, I made a multi-turn exam.

The multi-turn exam contains 60 items of two types: 20 list items and 40 story items. I used three generation seeds for the v3 models, so each item was answered three times with different sampling, and one seed was used for v2. For the list items, the user asks for a list (e.g. "name three animals"), the model makes the list, then the user asks first, second, last, or how many, and the result is scored deterministically against the model's list. This is done by converting an answer to lowercase and removing punctuation and articles, and a position answer counts as correct when it contains the asked-for item as a whole phrase and none of the other listed items. A how-many answer counts when it contains the number as a word or a digit.

In the story items, the user asks for a story, which is paired with three questions written in advance to fit the request, before the story exists. For example, the request "tell me a story about a girl who finds something in the garden" is paired with the questions "What color was it?", "What did she find?", and "Where did the story happen?". Say the model writes the story "Lily found a small key in the garden. She was happy", and is then asked the questions one-by-one. Three LLM judges, the same three models as the panels, label each answer using only the story as evidence. "A key" to the "what" question is supported, "At school" to the "where" question is contradicted, "Blue" to the color question is unsupported since color is not mentioned in the story, and "I don't know" is evasive. The label is the two-of-three majority among the judges.

The list results are shown in Table 7. Those rates count only replies that formed a proper list. Counting a reply with no list as wrong, bytes v3 scores 67, 6, 17, and 79% on first, second, last, and how many, and BPE v3 scores 52, 33, 44, and 74%, so the ordering between the two models is the same either way (Appendix A11). The incorrect answers lean toward the last item. For bytes v3, 18 of 33 wrong answers to "first" and "second" and 14 of 24 for BPE v3 (55 and 58%) named the last item in the list (Appendix A16). This is above the 40 to 42% expected if a wrong answer picked another item at random, but not to a significant extent ($p = 0.16$ and 0.09 on approximate two-sided binomial tests at the pooled chance rate, which averages over lists of three and four items).

On the story items, BPE v3 scored 36% on answers supported by its own story (Table 7). Among its answers that the story either supported or contradicted, 64% were supported. Bytes v3 and v2 scored about the same on supported answers in the story items at 16% and 18%, although among answers the story could check, v3's supported share was lower, 38% against 56%. The byte model's supported rate falls from 24% on the first question to 8% on the third question, which is consistent with the story leaving the window (Appendix A15). The story is still fully inside the

window when the first answer begins in every item, but the model crosses the window while responding, and when the third answer begins, part of the story has left the window in 26 to 32 of 40 items per seed, about a fifth of it on average, with one case where the story was lost entirely (Appendix A21). For BPE v3, supported answers fell from 42% on the first question to 29% on the third while the story remained within the 480-position window. The supported rate gap between bytes and BPE for the first question is not explained by the context window, as both models still hold the entire story when they begin to answer it, and the 17 bytes v3 first answers during which part of the story left the window were supported at the same 24% rate as the 103 where it stayed intact.

Metric	Bytes v2 (1 seed)	Bytes v3 (3 seeds)	BPE v2 (1 seed)	BPE v3 (3 seeds)
list: first	0%	79%	25%	64%
list: second	0%	7%	0%	43%
list: last	0%	21%	0%	81%
list: how many	33%	97%	0%	97%
story answers: supported	18%	16%	23%	36%
story answers: contradicted	14%	26%	18%	21%
story answers: unsupported	40%	38%	45%	29%
story answers: evasive	28%	19%	13%	13%
Supported among supported or contradicted (S/(S+C))	56%	38%	56%	64%

Table 7: Multi-turn exam. 60 items per run (20 list items, each with two position questions, and 40 story items with three questions each). List rows are scored deterministically against the first requested items of the list generated by the model (Appendix A11). Story rows are labeled by three judges with a two-of-three majority using the story text only. 10 of the 960 story answers had no majority label. They remain in the denominators but do not count in any of the four categories. “Supported among supported or contradicted” is supported divided by the sum of supported and contradicted. The labels describe the answer, not the question, so a question the story could have settled but the model dodged counts as unsupported or evasive here. v3 columns are averaged over the three generation seeds and v2 columns are one seed. Each list position is scored only on replies that make a proper list, 3 to 12 answers per run for v3 and 2 to 6 for v2 (Appendix A11). Rates are computed from unrounded seed means.

Section 6: Layers and Neurons

All of the base models were trained at six layers, but because BDH has all six layers sharing the same set of weights [13], the number of passes at inference can be varied [14]. I tested how varying the number of layers at inference from two to 16 affects the output quality. Among the tested depths (2, 3, 4, 5, 6, 7, 8, 12, 16) at every model size, inference at six layers had the lowest surprisal. At eight layers, the loss is about 0.3 nats worse, and by 12 the loss more than tripled (Table 8). For example, the prompt that produced “Dizzy was always dangerous. He liked to take big steps to stay safe” at six layers in the 100M byte model produced “criyIa 42622-24I0.d6A” at 12. The curves for 25M, 50M, and 100M are the same shape (Figure 2), so increasing the size does not make increasing passes more effective. The three BPE bases give the same shape, with the minimum at six layers (Appendix A18). To test whether the minimum follows the training depth, I trained a 25M byte model with four passes on the same data. Its minimum moved to four layers, 0.84 nats per byte versus 1.29 at six (Table 8). Recurrent-depth transformers can gain from extra iterations up to a point and then degrade, the “overthinking” reported in [15]. In a newer paper by Pathway, they introduced BDH-CQ, a model that answers by iterating in a latent workspace after the context has been read [18]. My depth result does not conflict with theirs, as their models were trained for variable depth while mine were only trained at one depth.

Layers at inference (trained at 6)	25M bytes	50M bytes	100M bytes
2	2.371	2.130	2.151
3	1.750	1.605	1.575
4	1.175	1.142	1.093
5	0.875	0.860	0.840
6	0.811	0.760	0.717
7	0.867	0.814	0.781
8	1.078	1.043	1.026
12	3.043	3.099	3.208
16	3.982	4.433	4.472

Table 8: Validation loss in nats per byte on the full 3 MB held-out text, every 512-position block scored, running each byte base model with a different number of passes through its shared layer at inference. Losses here are per byte, while Table 1 divides by characters, and the held-out text has 1.0055 bytes per character. After that conversion, the six-pass values agree with Table 1 to within 0.2%. The two scripts batch and align the 512-position blocks differently (this one scores the concatenated file in batches of 32 blocks, Table 1 scores each part separately), so the conversion is not exact. All models were trained with six layers. Lower is better. One seed used. The 25M model trained with four passes (Section 6) scored, at 2, 3, 4, 5, 6, 7, 8, 12 and 16 passes: 1.750, 1.041, 0.837, 0.926, 1.293, 2.018, 2.761, 3.966 and 3.865 nats per byte.

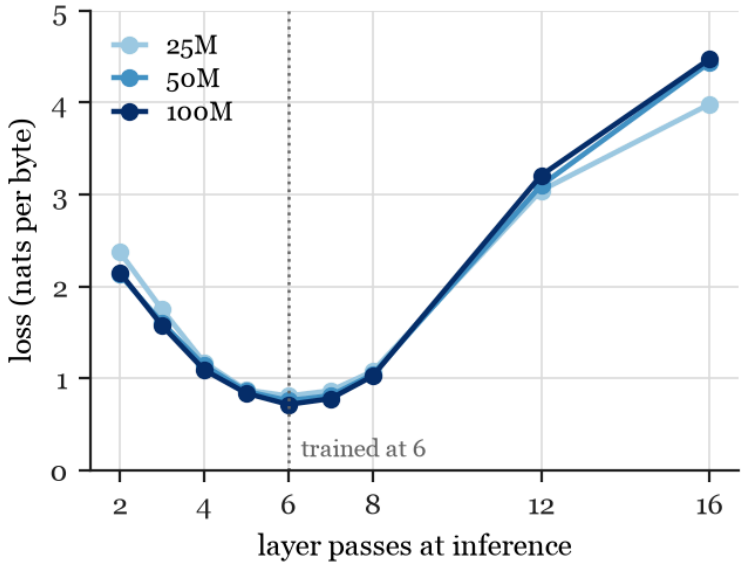


Figure 2: Validation loss against the number of layer passes at inference for the three byte base models, all trained with six passes (Table 8). One seed.

I also found neuron density to vary with model size. Density is the share of the 32·D neurons with a positive activation at a position, measured on 200,000 characters of training text and averaged over attention heads. For the byte model, the average percent of activated neurons falls from 13.5% at 25M to 11.2% at 100M. For BPE, it falls from 14.3% to 11.9% from the 25M to the 100M model (Table 9). So the larger models fire more neurons (about 1,100 at 25M, 1,800 at 100M) but it is a smaller share of the total, as reported for transformers [16]. Repeating the measurement on 200,000 characters of held-out text instead of the training text changes every model's density by up to three tenths of a point (Appendix A22). Untrained models of the same sizes fire about 50% of their neurons at every layer, so the sparsity is learned through training, not an inherent property of the architecture. Table 9 also shows that the first layer is the sparsest for bytes. This is not the case for BPE, where the first layer is one of the denser layers, and layer 6 is the densest. In every model, one attention head is sparser than the other three. Averaged over the six layers, the sparse head fires 5.6 to 11.2% of its neurons while the others fire 13 to 15%, a ratio of 1.9 to 2.3 in the byte models and 1.4 to 1.6 in the BPE models (Appendix A17). When I took the average of the largest 0.1% of each head's encoder weights, the sparse head's is the largest in five of the six models, 1.0 to 2.1 times the other heads' weights, and at 50M BPE one other head is 3% larger (Appendix A9). I have not yet determined why this occurs with sparse neuron firing, or what the sparser head responds to.

Model	Layer 1	Layer 2	Layer 3	Layer 4	Layer 5	Layer 6	Mean	Sparsest head	Densest head
25M bytes	9.3	13.1	14.4	14.7	14.5	14.9	13.5	5.2	18.3
50M bytes	8.7	12.8	13.6	14.1	14.3	14.3	13.0	3.8	18.0
100M bytes	7.7	12.1	12.1	11.9	11.7	11.7	11.2	3.7	17.8
25M BPE	15.6	12.2	13.5	13.7	14.8	15.9	14.3	8.9	17.5
50M BPE	12.7	11.0	11.7	11.9	12.7	13.5	12.3	7.3	16.5
100M BPE	12.0	10.8	11.3	11.6	12.3	13.3	11.9	7.5	14.7

Table 9: Percent of the neurons in a head with positive activation per position, averaged over the four heads, on 200,000 characters of training text. “Sparsest head” and “Densest head” are the extremes over the 24 layer-head pairs. One seed.

In order to see what individual neurons respond to I made a simple visualizer that shows, for each layer and head, every neuron’s firing rate, the contexts where it fired hardest, and the learned neuron-to-neuron wiring. I first labeled the 220 strongest-firing neurons. However, ranking by firing strength favors neurons that fire for a single token such as the end-of-text marker. So I decided to do a second pass looking at the most diverse-firing neurons in layers 4 to 6. If at least seven of a neuron’s eight strongest firings were on different tokens and it fired on 0.2% to 25% of all positions, the neuron was considered diverse. Out of these diverse neurons, I labeled the 160 strongest firing ones, and I found that 74 respond to grammatical structure, 51 are concept neurons that fire across word forms and pieces, 24 respond to the slot where a character is named, 9 respond to spelling, and 2 are unclear. Because shared weights make the same neuron appear at two or three layers with overlapping contexts, only 87 distinct ids appear among the 160. One neuron from the strength-ranked set fires on “saw”, and at layers 4 to 6 only after “One day, he” or “she”. Another, from the diverse set, fires on emotion adjectives across word pieces, such as “nervous”, “wor-ried”, “jeal-ous”, and “ash-amed”. Blindly labeling 400 more neurons showed that neurons firing on almost every position rarely got a confident label (7%) while selective ones often did (86%). A second blind labeling round compared 100 of the 160 selected neurons with 200 neurons chosen randomly from the entire model. 90% of the selected neurons got a confident label against 47% of the random neurons with firing rates in the same 0.2% to 25% band (Appendix A25). I used an LLM to label each neuron from eight contexts [17] and spot-checked the labels. I did not compare the results with an untrained model or test whether the labels predicted firing on new text.

Section 7: Limitations

The models I trained are small, from 25M to 100M parameters, and the largest learned from 5.4 GB of text. The small size shows while chatting with them. They seemed to have learned the shape of a fact before the fact itself, with statements such as “Water boils at 100°F” and “The capital is Frederice.” Story grounding is weak, with only 64% of the BPE v3 answers that the story could check being supported by it, and 38% for bytes (Section 5). The byte model’s 480-position sliding window severely limits conversation. In the exam the whole story is in the window when it starts the first answer, but the beginning of the story starts to leave it during the answers, and by the third answer, part of the story is gone in 26 to 32 of the 40 stories per seed (Section 5, Appendix A21). When the v3 models answer a list question incorrectly, they name the last item slightly more often than chance, but the counts are too small to confirm it is due to recency. v2 models were mostly unable to answer position questions about their own lists (Table 7). When chatting with the byte v2 model, it answered “Duck” to “first”, “last”, and “second” about a list it had just written.

Due to local compute and cost limitations, most of the measurements are single-run, with one training seed per model. I measured an additional seed for both the 25M and 50M pairs. At both sizes, the second seed changed combined bpc by at most 0.003 and the alphabet gap by 0.3 points or less, and every context band by at most 0.003 (Appendix A13). I did not retrain the 100M pair with a different seed. Batch shape differed between sizes. The 25M and 50M pairs trained on a 4090 with half the text per step compared to the byte 100M model, which trained on an H100, so size was not the only change between the 50M and 100M pairs. The two 100M models were trained on the same amount and mix of text (83% FineWeb-Edu and 17% TinyStories) but not identical documents, while the 25M and 50M pairs share their text exactly. This is because the byte model sampled documents from a larger text corpus.

The transformers were compared to the BDH models on the same number of shared-layer parameters and same learning rate, weight decay, and dropout. BDH applies its single set of weights six times per position, so it does about six times the compute per position as a transformer with the same number of parameters. At 25M, BDH took 7 hours compared to the transformer’s 0.9. At 50M it was 25 hours versus 4.2. At 100M, on the H100, BDH took 27 hours versus 4.1 for the transformer. The comparisons in this paper were matched on parameters, not compute, so the greater compute per position for BDH could have contributed to the edge over the transformer on bytes. In the BPE pairs, the BDH models also used about six times the compute per position and only tied the transformer (Appendix A8). The 100M BDH byte model also trained with twice the text per step compared to its corresponding transformer over the same total text, so the recipes did not match exactly at 100M. The transformers also ran with the learning rate and schedule chosen for BDH, with no attempt to find better settings for them, which may slightly favor BDH. On bytes, BDH outperformed them by about 5% in bpc at all

three sizes. On BPE, BDH was 0.8% behind the transformer at 25M, 1.0% ahead at 50M, and 2.4% ahead at 100M (Appendix A8). The alphabets are also not matched in total size. The BPE vocabulary matrices make the BPE models 8 to 16% larger than the byte models, and the transformers' wider residual stream makes their vocabulary matrices even larger, making the BPE transformer the largest model by total parameters (Appendix A8). The tokenizer created its vocabulary from the same corpus the models were trained on, which may have given BPE a small advantage. Only one vocabulary size, 8,192, was tried, so none of the alphabet results are generalized across tokenizers.

All five released tunes made the model a worse general language model, increasing combined held-out bpc by 23 to 57% for the byte models and 22 to 34% for BPE (Appendix A7, and 19 to 61% for the two retunes in Appendix A26), likely because the tuning data was never mixed in with the pretraining text. The byte model lost more than the BPE model for the v3 tunes (29% versus 22%), so part of the multi-turn gap in Section 5 could be caused by tuning rather than the context window. The tunes also differ from each other in exposure. In the same 2,000 steps, v1 saw each of its 20,000 examples about three times whereas v2 saw its 7,860 about eight times. v2 and v3 differed in steps, documents, and data pool size. The step-count control experiment (Section 5) shows that the steps mainly set the held-out cost. For the byte model the documents produced the recall ability at 128 characters, while for BPE it was produced by every tune. Opus 5 wrote the v2 and v3 tuning data, and it was also one of the three judges, so there was some risk of model bias. However, when repeating every panel with only the other two judges, the verdict was the same with v2 winning 93 to 20 and the four tie panels stayed at $p = 0.26$ to 1.0.

The models trained on 512-position blocks with no state carried between blocks, which differs from the original BDH paper [1], where the state is carried across 2,048-token minibatches. So, the block wall at 512 positions is partly a property of how I trained the models rather than the BDH architecture itself. The main planted-fact table used one template and one planted phrase per trial. The controls in Appendix A23 and A24 change the planting key and add a second phrase, but always ask the same question, so the test does not fully separate retrieval from learned repetition of the planted phrase. Also, recall does not decrease steadily with distance, as the BPE v3 scores show (Table 4).

In the multi-turn exam, the story questions were fit to the request rather than the story. Each model is also questioned about a story that the model itself wrote, so the story comparison between models mixes recall with the stories. Also, questions were asked in a fixed order every time, so part of the unsupported rate is questions the story never addressed. Because the order never changes, I cannot tell whether the later questions are harder in themselves or whether the drop in supported rate comes from less of the story remaining in the window. For the list items, each list position in v3 is scored on only 3 to 12 answers per run, so a flip in just one answer changes a single run's score by 8 to 33 points and the three-seed average by a third of that. Part

of v3's advantage on list items comes from v3 having trained on 360 dialogues with identical shapes to the exam's list questions. BPE v3's supported rate ranged from 32% to 41% across the three seeds.

Section 8: Next Steps

The 100M models can be retrained with a second seed, as that was only done on the 25M and 50M pairs. Additionally, the 100M model pair only shares the same amount and mix of text, not the same exact documents, so a fairer comparison can be made by training the BPE 100M on identical documents. For the context window, I only tested a size of 480, so future tests should include more sizes like 256 and 512 positions.

While I tuned the 100M BDH models, I only used base transformers of all sizes. Tuning a matched transformer on the v3 data will show if the recall improvement was specific to BDH. Also, to confirm the cause of each tuned model becoming worse at general language, each 100M model should be fine-tuned again with the pretraining data mixed into the tuning data.

Training larger models could help determine if the gap between BDH alphabets holds at around 7 to 8% (my prediction from Section 2), and if BDH keeps a 5% edge over a parameter-matched transformer on bytes. The models were trained on isolated 512-position blocks and restarting the context at each block costs the 100M byte model 0.05 to 0.08 bpc compared with using a window (Table 3, Appendix A4). Carrying the synaptic state across blocks during pretraining might improve the window results.

In the planted-fact test in Section 4, byte base models' gain is mostly on the letters after the first, and the first letter is rarely their top choice. Fine-tuning on question-style text improves this, but it is possible that including some of this in the pretraining data could provide this ability in a base model. As described in Section 6, each of the models has one attention head that is noticeably sparser than the others. Ablating that head could determine what it contributes to model predictions, and labeling the context in which its neurons fire could determine what it responds to.

References

1. Kosowski, A., Uznański, P., Chorowski, J., Stamirowska, Z., Bartoszkiewicz, M. *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain.* 2025. arXiv:2509.26507.
2. Penedo, G., Kydlíček, H., Ben Allal, L., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., Wolf, T. *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale.* 2024. arXiv:2406.17557.
3. Eldan, R., Li, Y. *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* 2023. arXiv:2305.07759.
4. Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., Raffel, C. *ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models.* 2021. arXiv:2105.13626.
5. Pagnoni, A., Pasunuru, R., Rodriguez, P., et al. *Byte Latent Transformer: Patches Scale Better Than Tokens.* 2024. arXiv:2412.09871.
6. Sennrich, R., Haddow, B., Birch, A. *Neural Machine Translation of Rare Words with Subword Units.* ACL 2016. arXiv:1508.07909.
7. Hugging Face. *Tokenizers* (software library). https://github.com/huggingface/tokenizers
8. Gigant, T., Peng, B., Quesnelle, J. *Decoupling the Benefits of Subword Tokenization for Language Model Training via Byte-level Simulation.* 2026. arXiv:2604.27263.
9. Schlag, I., Irie, K., Schmidhuber, J. *Linear Transformers Are Secretly Fast Weight Programmers.* 2021. arXiv:2102.11174.
10. Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.* 2020. arXiv:2006.16236.
11. Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M. *Efficient Streaming Language Models with Attention Sinks.* 2023. arXiv:2309.17453.
12. Arora, S., Eyuboglu, S., Timalsina, A., et al. *Zoology: Measuring and Improving Recall in Efficient Language Models.* 2023. arXiv:2312.04927.
13. Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., Kaiser, Ł. *Universal Transformers.* 2018. arXiv:1807.03819.
14. Geiping, J., McLeish, S., Jain, N., et al. *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach.* 2025. arXiv:2502.05171.
15. Kohli, H., Parthasarathy, S., Sun, H., Yao, Y. *Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers.* 2026. arXiv:2604.07822.
16. Li, Z., You, C., Bhojanapalli, S., et al. *The Lazy Neuron Phenomenon: On Emergence of Activation Sparsity in Transformers.* 2022. arXiv:2210.06313.
17. Bills, S., Cammarata, N., Mossing, D., et al. *Language Models Can Explain Neurons in Language Models.* OpenAI, 2023. https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html
18. Engdahl, B., Kosowski, A., Chorowski, J., Stamirowska, Z., Uznański, P., et al. *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning.* 2026. arXiv:2608.09888.

Appendix

The raw output behind every table is in the results folder of the code repository (github.com/d3vmeh/small-dragon-hatchling).

A1

Size	Text	Bytes bpc	BPE bpc	Bytes minus BPE	Gap % of byte score
25M	FineWeb-Edu part (2.5 MB)	1.2608	1.2110	0.050	3.9%
	TinyStories part (0.5 MB)	0.7642	0.7024	0.062	8.1%
	combined (3 MB)	1.1777	1.1259	0.052	4.4%
50M	FineWeb-Edu part (2.5 MB)	1.1833	1.0977	0.086	7.2%
	TinyStories part (0.5 MB)	0.7066	0.6293	0.077	10.9%
	combined (3 MB)	1.1035	1.0193	0.084	7.6%
100M	FineWeb-Edu part (2.5 MB)	1.1180	1.0390	0.079	7.1%
	TinyStories part (0.5 MB)	0.6660	0.5919	0.074	11.1%
	combined (3 MB)	1.0423	0.9641	0.078	7.5%

Table A1: Table 1 split by held-out text source. Both parts show BPE ahead at every size. One training seed. The file is built by build_eval_text.py in the repository from FineWeb-Edu shard 013 (training used shards 000 to 005) and the TinyStories validation split, both in file order.

A2

Model	[8,16)	[16,32)	[32,64)	[64,128)	[128,256)	[256,512)
bytes_25m	1.4762	1.3016	1.1927	1.1186	1.0669	1.0389
bytes_50m	1.4456	1.2537	1.1300	1.0477	0.9924	0.9623
bytes_100m	1.4157	1.2125	1.0789	0.9909	0.9333	0.9021
bpe_25m	1.2067	1.1293	1.0751	1.0439	1.0298	1.0264
bpe_50m	1.1311	1.0431	0.9821	0.9460	0.9278	0.9218
bpe_100m	1.0887	0.9961	0.9328	0.8948	0.8753	0.8683
gpt_25m	1.5137	1.3575	1.2613	1.1925	1.1381	1.1032
gpt_50m	1.4758	1.2972	1.1838	1.1057	1.0494	1.0150
gpt_100m	1.4503	1.2604	1.1377	1.0541	0.9967	0.9625
gpt_bpe_25m	1.2265	1.1463	1.0860	1.0466	1.0254	1.0152
gpt_bpe_50m	1.1521	1.0653	1.0015	0.9615	0.9401	0.9297
gpt_bpe_100m	1.1183	1.0273	0.9620	0.9217	0.9000	0.8894
bytes_25m_s2	1.4789	1.3025	1.1929	1.1180	1.0663	1.0387
bpe_25m_s2	1.2077	1.1308	1.0771	1.0462	1.0322	1.0291
bytes_50m_s2	1.4482	1.2553	1.1306	1.0474	0.9914	0.9611
bpe_50m_s2	1.1303	1.0427	0.9815	0.9454	0.9281	0.9222

Table A2: bpc by context-length band (positions; blocks of 2k with loss on the last k), 1 MB FineWeb-Edu + 0.5 MB TinyStories held-out. Table 2 is the matched-character subset. gpt_* = transformers; *_s2 = second training seed. On this same 1.5 MB text the block-averaged (Table 1 style) scores of the 100M pair are bytes 0.9714 and BPE 0.8927, a gap of 8.1% (results_100m_subset1p5mb.txt in the repository).

A3

Model / setting	Context	Recompute ms per position	Streaming ms per position	Note
57M pilot, fp32 Strix Halo iGPU	128	80.6	45.3	1.8x
	256	144.1	45.6	3.2x
	512	297.7	45.8	6.5x
	1,024	884.1	45.3	19.5x
	2048	2682.4	45.5	59.0x
	4096	7916.1	45.5	174.0x
100M bytes, fp32 iGPU (test_window_long)	4096 stream	439	91	no-window 73 ms/pos

Table A3: cost per new position. Streaming is flat; recompute grows with context. The 45 ms figure is the earlier 57M model; the 100M in fp32 costs 73 ms without the window and 91 ms with it, 439 ms for recompute-on-last-480 (456 ms for the BPE model). Batch 1, fp32. The 57M figures are the mean of five timed positions after one warm-up, and the 100M figures are wall-clock time over 4,096 positions divided by the position count.

A3b

Checkpoint	Alphabet	D	Positions	Max logit diff	Mean logit diff	Next 40 greedy symbols
ckpt_big.pt	bytes	384	512	1.91e-05	2.47e-06	IDENTICAL
ckpt_h100.pt	bytes	512	512	2.00e-05	1.97e-06	IDENTICAL
ckpt_100m_bpe.pt	bpe8k	512	512	1.62e-05	1.28e-06	IDENTICAL

Table A3b: streaming vs full recompute on the same 512 positions, three models. Differences are float reordering noise.

A4

Alphabet	Chunk (positions)	Window minus recompute-480: mean	Sd over passages	Max gap any passage	No-window / fresh-block: mean	Min	Max	Window minus fresh-block: mean
bytes	0-512	-0.0002	0.0010	0.002				
	512-1024	0.0021	0.0063	0.012	3.41	3.10	3.94	-0.060
	1024-1536	-0.0019	0.0034	0.007	3.36	2.78	3.75	-0.068
	1536-2048	0.0027	0.0066	0.019	3.73	3.04	4.92	-0.047
	2048-2560	0.0070	0.0099	0.025	3.66	3.22	4.12	-0.076
	2560-3072	0.0027	0.0044	0.008	3.81	2.69	4.77	-0.070
	3072-3584	0.0038	0.0088	0.021	3.93	3.08	4.74	-0.077
	3584-4096	0.0035	0.0065	0.013	3.87	3.31	4.60	-0.054
bpe8k	0-512	-0.0001	0.0003	0.001				
	512-1024	0.0020	0.0021	0.006	1.55	1.42	1.65	-0.023
	1024-1536	0.0020	0.0019	0.004	1.55	1.43	1.64	-0.023
	1536-2048	0.0021	0.0024	0.006	1.61	1.50	1.88	-0.028
	2048-2560	0.0029	0.0036	0.009	1.78	1.48	2.33	-0.022
	2560-3072	0.0030	0.0029	0.009	1.97	1.55	2.42	-0.025
	3072-3584	0.0055	0.0057	0.021	2.00	1.56	2.45	-0.025
	3584-4096	0.0060	0.0145	0.047	1.91	1.64	2.36	-0.033

Table A4: the window test over ten 4,096-position passages (byte offsets 0, 40k .. 360k of the held-out file), 100M bases. Table 3 shows the offset-0 passage.

A5

Arm	bpc	Steps	Train val
bytes	1.3855	13272	0.9834 nats/byte
BPE-8k	1.2107	13175	3.3653 nats/token
gap (bytes minus BPE)	0.1748		12.6% of the byte score

Table A5: the earlier 57M pilot at matched COMPUTE (120 min each on the 4090, micro 8 x accum 4, block 512): BPE read ~4x the text in the same time. Train val is each run's own validation loss on the last 5% of its training text, in nats per byte or per token, not the held-out text.

A6

Model	Distance (chars)	Needle nats	Baseline nats	Gain	Recall
100M base (3 trials)	0	1.70	11.42	9.72	3/3
	64	2.92	14.84	11.93	3/3
	128	3.65	13.53	9.88	1/3
	256	6.16	15.75	9.59	0/3
	384	5.06	15.26	10.20	0/3
	600	27.34	28.83	1.49	0/3
	1,024	21.99	23.05	1.06	0/3
chat v1 (3 trials)	0	2.83	11.79	8.96	1/3
	64	4.97	15.22	10.25	0/3
	128	4.68	11.97	7.30	0/3
	256	6.00	14.92	8.92	0/3
	384	5.48	15.16	9.67	0/3
	600	40.28	37.34	-2.94	0/3
	1,024	24.63	26.13	1.50	0/3
chat v2 (3 trials)	0	5.03	14.25	9.22	0/3
	64	6.81	25.19	18.38	0/3
	128	4.81	15.49	10.68	0/3
	256	6.25	19.38	13.13	0/3
	384	5.56	19.94	14.38	0/3
	600	36.66	35.75	-0.90	0/3
	1,024	25.47	30.30	4.82	0/3
chat v3 (3 trials, the 2/3-at-384 pilot)	0	0.71	11.11	10.40	3/3
	64	4.06	17.84	13.79	0/3
	128	0.41	13.15	12.74	3/3
	256	2.38	15.66	13.28	2/3
	384	1.11	16.49	15.38	2/3
	600	46.37	47.98	1.61	0/3
	1,024	28.25	32.07	3.82	0/3

Table A6: the early three-word probe runs (banana, purple, and rocket, no window) cited in Section 4: v1 recalls less than the base, v2 raises gain to 18 nats with no recall in these three trials (30-trial run of the released v2 models in A27), v3 pilot 2/3 at 384 (2 of 30 in the recorded run, Table 4).

A7

Tune	Text	Bytes bpc	Vs byte base	BPE bpc	Vs BPE base
v1	fineweb	1.3741	+23%		
	tinystories	0.8241	+24%		
	combined	1.2820	+23%		
v2	fineweb	1.7192	+54%	1.3662	+31%
	tinystories	1.2179	+83%	0.9497	+60%
	combined	1.6353	+57%	1.2965	+34%
v3	fineweb	1.4247	+27%	1.2455	+20%
	tinystories	0.9215	+38%	0.8089	+37%
	combined	1.3404	+29%	1.1723	+22%

Table A7: held-out bpc of the chat-tuned 100M models on the Table 1 text (bases: bytes 1.042, BPE 0.964 combined). v1 has no BPE arm.

A8

Arm / size	Bytes 25M	Bytes 50M	Bytes 100M	BPE 25M	BPE 50M	BPE 100M
architecture	rotary GPT, 8 layers, 8 heads, d=512	rotary GPT, 8 layers, 12 heads, d=768	rotary GPT, 8 layers, 16 heads, d=1024	rotary GPT, 8 layers, 8 heads, d=512	rotary GPT, 8 layers, 12 heads, d=768	rotary GPT, 8 layers, 16 heads, d=1024
params (M)	25.45	57	101.2	33.6	69.2	117.5
param accounting	25.45M	57.0M	101.2M	25.2M core + 8.4M vocab	56.6M core + 12.6M vocab	100.7M core + 16.8M vocab
text	1.37 GB	3.08 GB	5.47 GB	1.36 GB	3.06 GB	5.44 GB
steps	83,865	188,006	333,618	21,671	48,749	86,655
hardware, time	4090, 0.9 h	4090, 4.2 h	H100, 4.1 h	4090, 0.29 h	4090, 1.06 h	4090, 3.03 h
transformer bpc (combined)	1.2447	1.1565	1.1065	1.1174	1.0291	0.9876
BDH bpc (same text)	1.1777	1.1035	1.0423	1.1259	1.0193	0.9641
BDH edge	+5.4% (BDH better)	+4.6%	+5.8%	-0.8% (transformer better)	+1.0%	+2.4%

Table A8: the six transformer controls (gpt.py: pre-norm, rotary positions, untied lm_head, no biases, dropout 0.1), trained with train_scale.py on the same corpus and recipe as the BDH pairs. The text budget is 54 bytes per core parameter of each model, so the transformers, with slightly more core parameters, read up to 0.6% more text than their BDH twins (bytes 83,865 vs 83,376 steps at 25M and 188,006 vs 187,272 at 50M, BPE 21,671 vs 21,656 and 48,749 vs 48,726). At 100M the match is looser: the byte transformer was trained on the same amount and mix of text but not the same documents, the BDH byte-100M trained with twice the text per step (see A10), and the BPE transformer ran 86,655 steps compared to BDH's 86,000. Untuned BDH hyperparameters throughout. Held-out bpc on the Table 1 text.

A9

Model	Sparsest head	99.9th-percentile encoder weight: sparse vs other three	Ratio range	Mean of largest 0.1%: sparse vs others	Ratio range
25M bytes	h2	0.198 vs 0.119, 0.120, 0.140	1.41-1.67	0.238 vs 0.130, 0.133, 0.167	1.42-1.83
50M bytes	h3	0.144 vs 0.098, 0.099, 0.103	1.40-1.46	0.164 vs 0.114, 0.112, 0.117	1.41-1.47
100M bytes	h1	0.132 vs 0.114, 0.119, 0.101	1.11-1.30	0.152 vs 0.135, 0.149, 0.116	1.02-1.31
25M BPE	h1	0.307 vs 0.161, 0.176, 0.165	1.74-1.90	0.375 vs 0.182, 0.208, 0.188	1.80-2.06
50M BPE	h1	0.234 vs 0.227, 0.144, 0.156	1.03-1.62	0.279 vs 0.288, 0.162, 0.182	0.97-1.73
100M BPE	h2	0.210 vs 0.122, 0.127, 0.135	1.55-1.73	0.245 vs 0.137, 0.143, 0.160	1.54-1.80

Table A9: encoder-weight tail of the sparsest head (Appendix A17) vs the other three heads. By the 99.9th percentile the sparse head is largest in 6 of 6; by the mean of the largest 0.1% in 5 of 6 (at 50M BPE one other head is 3% larger). One seed.

A10

Model	BDH 25M / 50M pairs (bytes, BPE)	BDH byte-100M	BDH BPE-100M	BDH 25M at 4 passes	Transformers	Chat tunes v1, v2	Chat tunes v3 (bytes, BPE)
layers / heads	6 shared, 4 heads, N=32D	6 shared, 4 heads, N=16,384	6 shared, 4 heads, N=16,384	4 shared	8 layers, d/64 heads, rotary	-	-
block	512	512	512	512	512	512	512
batch per step	32 seqs = 16,384 atoms	64 seqs = 32,768 bytes	32 seqs = 16,384 tokens	32 seqs	32 seqs (100M bytes: on H100)	32 (micro 8 x 4)	32
text	54 bytes per core param (1.37 / 3.07 GB)	5.41 GB (sampled blocks from the 13 GB pod corpus)	1.41B tokens (~5.4 GB) from the 6 GB lab corpus	1.37 GB	54 bytes per core param	v1 pool 20,000; v2 7,860 (500 held out)	6,960 (500 held out)
optimizer	AdamW, wd 0.1	AdamW, wd 0.1	AdamW, wd 0.1	AdamW, wd 0.1	AdamW, wd 0.1	AdamW, wd 0.01	AdamW, wd 0.01
learning rate	1e-3 -> 1e-4 cosine, warmup 1,500	1e-3 -> 1e-4 cosine, warmup 2,000	1e-3 -> 1e-4 cosine, warmup 1,500	same	same	1e-4 flat	1e-4 flat
dropout	0.1	0.1	0.1	0.1	0.1	0.1	0.1
precision	bf16 autocast	bf16 autocast	bf16 autocast	bf16	bf16	bf16	bf16
steps	25M: 83,376 / 21,656; 50M: 187,272 / 48,726	165,000	86,000	83,376	see A8	2,000	800
hardware, wall-clock	4090: 7.1 h / 1.9 h; 25.3 h / 7.4 h	H100, ~27 h	4090, 19.8 h	4090	see A8	Bytes: H100, 24 min BPE v2: 4090, 20.5 min	Bytes: 4090, 25 min BPE: Strix Halo, 51 min

Table A10: pretraining and tuning recipes. All BDH models share the paper architecture (6 shared-weight layers, 4 heads, N = 32D per head), except the 4-pass model. Scripts: train_scale.py for the 25M and 50M pairs, the 4-pass model (LAYERS=4) and the six transformers (MODEL=gpt), train_h100.py for the byte-100M, train_100m.py for the BPE-100M, and tune_spans.py for every chat tune. Loss only on assistant spans for the tunes.

A11

Run	List items	Replies that formed a proper list	First scored	Second scored	Last scored	How many scored
bpe_v2 s0	20	8	4	5	2	5
bpe_v3 s0	20	13	6	9	3	8
bpe_v3 s1	20	16	8	9	3	12
bpe_v3 s2	20	16	8	10	4	10
bytes_v2 s0	20	9	6	4	2	6
bytes_v3 s0	20	16	8	10	4	10
bytes_v3 s1	20	17	7	10	6	11
bytes_v3 s2	20	17	8	10	5	11

Table A11: how many list answers were actually scored per run (only replies that parsed as a list of the requested length are scored; each item carries two position questions). A reply counts as a list when it contains no sentence break and splits on commas, “and”, “or”, line breaks, or bullets into at least the requested number of items, of which the first ones are kept after dropping articles and punctuation. One answer moves a score by 8 to 50 points. Scored on every question asked instead, bytes v3 gets 67, 6, 17, and 79% on first, second, last, and how many and BPE v3 52, 33, 44, and 74%. Of the 112 scored list replies across the eight runs, one (bytes v2) listed more items than asked, and scoring every reply against its full list instead changes no rate in Table 7 (results_exam_list_truncation.txt in the repository).

A12

Panel	Stories (40)	Questions (40)	Explanations (24)	Micro-tasks (16)	Note
v1 vs v2	story 25-14 (1 none)	QA 39-1	explain 19-5	micro 14-2	p (story) 0.11
bytes v2 vs BPE v2	story 21-19 (BPE)	QA 16-16 (8 none)	explain 13-10	micro 7-2 (7 none)	
bytes v3 vs BPE v3	story 20-20	QA 17-16	explain 12-9 (bytes)	micro 5-3 (bytes)	
BPE v2 vs v3	story 22-17 (v3)	QA 15-14 (v3, 11 none)	explain 14-9 (v2)	micro 6-1 (v2)	
v2 vs v3 (bytes)	story 23-17 (v2)	QA 17-15 (v3, 8 none)	explain 14-8 (v3)	micro 6-2 (v3, 8 none)	

Table A12: per-category wins by two-of-three majority. Categories are assigned by a keyword rule over the prompt text (40/40/24/16); the prompt file's own grouping is 40/32/24/24. In each cell the winner's count comes first and the winner is named in parentheses. Unnamed cells went to the panel's overall winner (Table 6), and 16-16 is a tie.

A13

Size	Text	Bytes seed 1	Bytes seed 2	BPE seed 1	BPE seed 2	Gap seed 1	Gap seed 2
25M	fineweb	1.2608	1.2595	1.2110	1.2144	3.9%	3.6%
25M	tinystories	0.7642	0.7658	0.7024	0.7032	8.1%	8.2%
25M	combined	1.1777	1.1768	1.1259	1.1288	4.4%	4.1%
50M	fineweb	1.1833	1.1810	1.0977	1.0979	7.2%	7.0%
50M	tinystories	0.7066	0.7057	0.6293	0.6286	10.9%	10.9%
50M	combined	1.1035	1.1015	1.0193	1.0193	7.6%	7.5%

Table A13: the 25M and 50M pairs retrained with a second seed (init and batch order). Combined held-out bpc moved by at most 0.003 (one part-score, 25M BPE on FineWeb, by 0.0034) and the alphabet gap by 0.3 points at 25M and 0.1 at 50M; every context band (A2, *_s2 rows) moved by at most 0.003 at both sizes.

A14

Model	Distance (chars)	Total gain (nats)	Gain on the word's first atom	Gain on the remaining atoms	Recall
25M bytes	0	10.32	2.64	7.34	6/10
	64	9.91	1.21	8.30	1/10
	128	9.00	1.11	7.20	0/10
	256	7.60	0.68	6.30	0/10
	384	6.66	0.47	5.82	0/10
	600	-0.85	-0.05	-0.70	0/10
	1,024	0.48	0.05	-0.07	0/10
50M bytes	0	9.22	2.70	6.45	10/10
	64	8.66	1.24	6.79	0/10
	128	8.62	1.38	6.80	0/10
	256	7.84	0.81	6.46	0/10
	384	7.09	0.62	6.09	0/10
	600	3.89	0.13	3.74	0/10
	1,024	2.99	0.04	2.64	0/10
100M bytes	0	10.19	2.45	7.23	10/10
	64	11.51	1.59	8.44	8/10
	128	9.27	1.41	7.41	2/10
	256	8.22	0.39	7.04	0/10
	384	8.35	0.64	6.98	0/10
	600	1.48	-0.35	2.72	0/10
	1,024	0.83	0.09	0.65	0/10
100M bytes v3	0	11.39	2.85	8.50	9/10
	64	13.74	1.23	12.37	1/10
	128	12.50	2.81	9.66	9/10
	256	12.39	1.41	10.40	4/10
	384	12.17	1.39	10.70	2/10
	600	0.85	0.23	1.67	0/10
	1,024	2.94	0.33	1.65	0/10
100M BPE	0	10.80	7.55	2.35	9/10
	64	9.10	6.09	2.59	2/10
	128	9.78	6.90	2.36	7/10
	256	9.20	5.76	2.85	5/10
	384	8.81	5.00	2.93	3/10

Model	Distance (chars)	Total gain (nats)	Gain on the word's first atom	Gain on the remaining atoms	Recall
100M BPE	600	6.72	3.33	2.69	0/10
	1,024	5.42	1.94	2.75	0/10
100M BPE v3	0	11.96	8.10	2.15	10/10
	64	11.72	8.08	3.51	10/10
	128	11.46	8.47	2.86	10/10
	256	12.52	8.85	3.61	9/10
	384	13.67	9.49	3.79	9/10
	600	13.17	9.31	3.75	10/10
	1,024	12.01	7.80	4.01	7/10

Table A14: the planted-fact gain split by atom. For byte models the first atom is the word's first letter (after the space) and the rest are its remaining letters; for BPE the first atom is the word's first token (the ten words are one to three tokens in the 8k vocabulary) and the rest are its remaining tokens. The final period is not counted in either part, for both alphabets. The total gain does include the period, so the two parts do not sum to the total. Ten words, one filler slice, no window.

A15

Model	Supported Q1	Supported Q2	Supported Q3	Decided-correct Q1	Q2	Q3
bytes v3 (3 seeds)	24%	15%	8%	51%	38%	21%
BPE v3 (3 seeds)	42%	38%	29%	65%	65%	61%
bytes v2 (1 seed)	23%	15%	18%	60%	46%	64%
BPE v2 (1 seed)	23%	23%	23%	64%	45%	64%

Table A15: story-item exam rates by question number (two-of-three judge majority; 120 answers per column for v3, 40 for v2). The byte model falls as the story leaves its window; the gap is present on Q1 already.

A16

Model	Wrong first/second answers	Named the last item	Share	Chance (1/(n-1), lists of 3-4)
bytes v3	33	18	55%	42%
BPE v3	24	14	58%	40%
bytes v2	10	1	10%	48%
BPE v2	8	2	25%	48%

Table A16: recency check. Chance = a wrong answer naming another item at random; v3 models sit modestly above it.

A17

Model	Sparsest head, mean over layers (%)	Other three heads, mean over layers (%)	Ratio (other three / sparsest)
25M bytes	8.0	15.3	1.9
50M bytes	7.6	14.7	1.9
100M bytes	5.6	13.1	2.3
25M BPE	11.2	15.3	1.4
50M BPE	8.5	13.5	1.6
100M BPE	8.7	13.0	1.5

Table A17: density per head averaged over the six layers: the lowest-density head, the mean of the other three, and their ratio. Same measurement as Table 9 (200,000 characters of training text). One seed.

A18

Layer passes at inference (trained at 6)	BPE 25M (nats/token)	BPE 50M	BPE 100M
2	5.260	4.846	4.659
3	4.653	4.437	4.332
4	3.849	3.704	3.533
5	3.213	2.959	2.879
6	2.999	2.715	2.569
7	3.159	2.883	2.769
8	3.537	3.295	3.125
12	6.094	4.810	4.643
16	8.215	6.894	6.548

Table A18: the layer-pass experiment on the three BPE base models. Units are nats per token, not comparable to Table 8 or Table 1. Each model has its minimum at six passes, the depth it was trained at, on the nine-point grid. Lower is better. One seed.

A19

Distance (chars)	Transformer 25M	BDH 25M base	Transformer 50M	BDH 50M base	Transformer 100M	BDH 100M base
0	11.7	10.3	11.1	9.2	11.7	10.2
64	11.9	9.6	12.6	8.8	13.1	11.1
128	11.2	9.4	12.2	8.7	12.7	9.7
256	10.5	8.2	12.0	7.6	12.8	8.7
384	9.0	6.8	10.3	7.4	11.9	8.8
600	0.0	-1.1	0.0	2.4	0.0	1.2
1,024	0.0	0.4	0.0	2.4	0.0	1.3

Table A19: gain (nats of surprisal removed by the planted phrase, mean of 30 trials) for byte transformers of matched size and the BDH byte bases, by distance in characters. Past 512 characters the transformers score by recompute within the block so their gain is zero, while the BDH bases read past the block through synaptic state. One training seed; ten recorded words in three filler texts.

A20

Distance (chars)	Transformer 25M	BDH BPE 25M base	Transformer 50M	BDH BPE 50M base	Transformer 100M	BDH BPE 100M base
0	9.0	6.1	11.5	9.8	11.8	10.8
64	10.0	5.5	9.8	9.1	11.6	10.5
128	9.5	4.8	9.4	8.0	11.6	9.2
256	9.0	3.6	9.9	7.4	11.9	8.9
384	9.1	3.3	10.5	6.2	11.5	8.3
600	8.1	2.4	8.9	5.5	10.7	7.5
1,024	5.7	1.3	5.6	4.1	6.9	5.4

Table A20: gain (nats, mean of 30 trials) for BPE transformers of matched size and the BDH BPE bases. For both, 1,024 characters is still inside the 512-token block, so no distance exceeds either model's context. BDH rows are no-window streaming. One training seed; ten recorded words in three filler texts.

A21

Run	Median story length (atoms)	Story partly gone, Q1	Story partly gone, Q2	Story partly gone, Q3	Mean share gone, Q1	Mean share gone, Q2	Mean share gone, Q3
bytes v2, seed 0	352	0/40	22/40	30/40	0%	5%	21%
bytes v3, seed 0	341	0/40	19/40	27/40	0%	7%	23%
bytes v3, seed 1	353	0/40	23/40	32/40	0%	5%	22%
bytes v3, seed 2	335	0/40	17/40	26/40	0%	4%	19%
BPE v2, seed 0	99	0/40	0/40	0/40	0%	0%	0%
BPE v3, seed 0	95	0/40	0/40	0/40	0%	0%	0%
BPE v3, seed 1	101	0/40	0/40	0/40	0%	0%	0%
BPE v3, seed 2	101	0/40	0/40	0/40	0%	0%	0%

Table A21: how much of the model's own story has left the 480-position window during the exam, from the positions exam.py records after every reply and the length of the story reply. A story counts as partly gone when at least one of its atoms is outside the window at the moment the answer begins, and the share is the fraction of the story's atoms already outside. What leaves first is the opening of the story. 40 story items per run. For the byte models the story is complete when the first answer begins in every item, and by the third answer part of it has gone in 26 to 32 of 40 items, on average about a fifth. One story in the four byte runs was lost entirely (bytes v3, seed 0, by the end of the second answer). The BPE stories are about 100 tokens and never reach the window.

A22

Model	Training text (Table 9)	Held-out text	Untrained, same size	Untrained, per-layer range
25M bytes	13.5	13.39	49.91	49.8 to 50.1
50M bytes	13.0	12.73	50.02	50.0 to 50.0
100M bytes	11.2	11.21	49.99	49.9 to 50.0
25M BPE	14.3	14.07	50.02	50.0 to 50.1
50M BPE	12.3	12.07	49.99	50.0 to 50.0
100M BPE	11.9	11.79	50.04	50.0 to 50.1
25M bytes, seed 2	not measured	13.48	as seed 1	as seed 1
50M bytes, seed 2	not measured	12.22	as seed 1	as seed 1
25M BPE, seed 2	not measured	13.61	as seed 1	as seed 1
50M BPE, seed 2	not measured	12.43	as seed 1	as seed 1

Table A22: neuron density measured on 200,000 characters of HELD-OUT text (the Table 1 evaluation text) instead of training text, and on an untrained model of the same shape. Density is the share of the 32·D neurons per head with a positive activation at a position, averaged over the four heads. The seed-2 rows (2026-09-29) are the second-seed 25M and 50M bases on the same held-out text. Each again has one head sparser than the other three, at a different head index than in seed 1. The other three heads are on average 1.8, 2.3, 1.5 and 1.3 times denser than that head (bytes 25M, bytes 50M, BPE 25M, BPE 50M), compared to 1.9, 2.0, 1.3 and 1.7 for seed 1 on this text (A17's 1.9, 1.9, 1.4 and 1.6 are on training text).

A23

Model	Distance (chars)	Gain: matched → mismatched	Recall /30: matched → mismatched	First atom: matched → mismatched
100M bytes	0	10.2 → 10.0	30 → 15	2.45 → 2.28
	64	11.1 → 9.7	17 → 0	1.59 → 0.80
	128	9.7 → 8.7	5 → 0	1.41 → 0.72
	256	8.7 → 8.0	0 → 0	0.39 → 0.27
	384	8.8 → 7.9	0 → 0	0.64 → 0.32
	600	1.2 → 1.5	0 → 0	-0.35 → -0.41
	1,024	1.3 → 1.2	0 → 0	0.09 → 0.06
100M bytes v3	0	11.4 → 11.1	27 → 21	2.85 → 2.40
	64	13.9 → 12.4	20 → 12	1.23 → -0.67
	128	14.3 → 12.5	28 → 11	2.81 → 1.40
	256	12.6 → 11.9	4 → 1	1.41 → 0.74
	384	13.5 → 12.3	2 → 0	1.39 → 0.39
	600	0.2 → 0.9	0 → 0	0.23 → 0.04
	1,024	2.5 → 1.8	0 → 0	0.33 → 0.26
100M BPE	0	10.8 → 9.1	27 → 3	7.55 → 6.14
	64	10.5 → 8.9	17 → 9	6.09 → 5.48
	128	9.2 → 8.1	16 → 8	6.90 → 5.01
	256	8.9 → 7.0	10 → 1	5.76 → 3.30
	384	8.3 → 6.4	6 → 0	5.00 → 2.74
	600	7.5 → 5.9	4 → 0	3.33 → 2.37
	1,024	5.4 → 4.6	0 → 0	1.94 → 1.30
100M BPE v3	0	12.0 → 10.5	30 → 27	8.10 → 6.91
	64	13.1 → 13.1	29 → 29	8.08 → 8.17
	128	12.0 → 12.0	28 → 29	8.47 → 8.32
	256	12.8 → 12.2	28 → 25	8.85 → 7.90
	384	12.0 → 11.9	20 → 23	9.49 → 9.39
	600	12.5 → 11.3	27 → 20	9.31 → 8.52
	1,024	10.2 → 7.2	10 → 2	7.80 → 4.71

Table A23: the mismatched-key control. The word is planted under a different key ("The dog's name is X.") while the question still asks for the secret word, so plant and cue share no key wording. Every column uses the ten recorded words in the same three filler texts, so matched and mismatched are paired. For the two v3 models, the matched columns are the same recorded-word runs as Table 4. Three filler texts, 30 trials per cell for gain and recall, no window. The first-atom columns are one filler slice (10 trials) and give the word's first letter for byte models, the word's first token for BPE.

A24

Model	Distance (chars)	Gain on the asked word (nats)	Said the asked word /30	Said the distractor /30
100M bytes	0	9.6	12	12
	64	11.1	6	16
	128	9.8	2	4
	256	8.9	0	0
	384	8.9	0	0
	600	2.9	0	0
	1,024	0.5	0	0
100M BPE	0	9.2	12	6
	64	9.7	12	6
	128	8.8	10	7
	256	8.6	10	5
	384	8.2	4	1
	600	7.5	3	2
	1,024	5.6	0	0
100M bytes v3	0	9.8	9	12
	64	14.0	16	9
	128	14.0	22	7
	256	13.4	4	2
	384	13.8	10	6
	600	3.8	0	0
	1,024	3.5	0	0
100M BPE v3	0	9.6	15	3
	64	12.6	25	4
	128	11.3	22	7
	256	12.2	21	7
	384	11.7	18	6
	600	12.0	24	4
	1,024	10.5	8	4

Table A24: the two-plant control. Two words are planted in one context under two keys, “The secret word is X.” and “The dog’s name is Y.” (which one comes first alternates by trial, and Y is the next word in the ten-word list so that each word serves once as X and once as Y per filler), followed by filler of the stated length and the cue “The secret word is”. Retrieval by key predicts X; naming the odd word predicts Y about as often as X. Ten recorded words in three filler texts, 30 trials per cell, no window. Gain is on X. Gain is measured against the same filler with both planting sentences removed.

A25

Round	Group	N	High	Medium	Low	High share
1 (2026-09-12)	firing rate >= 90% of positions	323	24	132	167	7%
1 (2026-09-12)	firing rate < 90%	77	66	8	3	86%
2 (2026-09-28)	selected by the filter (100 of the 160)	100	90	10	0	90%
2 (2026-09-28)	uniform random sample, all neurons	200	97	76	27	49%
2 (2026-09-28)	uniform random sample, rate 0.2 to 25% only	167	79	67	21	47%
2 (2026-09-28)	uniform random sample, rate >= 90%	2	0	1	1	0%

Table A25: blind labeling rounds on the BPE-100M base. Each neuron was labeled by an LLM from its eight strongest firing contexts, with a confidence tag: high when one pattern covers seven or eight of the eight contexts. Round 1 (400 neurons from the curated export, split by firing rate afterwards) shows that always-on neurons rarely get a confident label. Round 2 mixed 100 of the 160 selected neurons with 200 neurons drawn uniformly from all 6 x 4 x 16,384, shuffled so the labelers did not know the group. Restricting the random group to the filter's own firing-rate band leaves the gap at 90% against 47%.

A26

Tune	Steps	Bytes bpc (combined)	Vs byte base	BPE bpc (combined)	Vs BPE base	Final train loss, nats per byte / nats per token
v2 data (Table 6, A7)	2,000	1.6353	+57%	1.2965	+34%	0.045 / 0.045
v2 data, retuned	800	1.2982	+25%	1.1497	+19%	0.105 / 0.327
v3 data (Table 6, A7)	800	1.3404	+29%	1.1723	+22%	0.133 / 0.399
v3 data, retuned	2,000	1.6802	+61%	1.3463	+40%	0.021 / 0.051
Panel (120 prompts, equal steps)	**v3 wins**	**v2 wins**	**Undecided**	**p (exact binomial on decided)**	**Scoring**	
bytes, 800 steps	45	47	28	0.92	two-of-three majority	
bytes, 2,000 steps	52	54	14	0.92	two-of-three majority	
BPE, 800 steps	46	51	23	0.68	two-of-three majority	
BPE, 2,000 steps	50	48	22	0.92	two-of-three majority	

Table A26: the step-count control. The v2 and v3 tuning sets were each retuned from the same bases at the other set's step count (v2 for 800 steps, v3 for 2,000), same batch of 32 and learning rate. Top: held-out bpc on the Table 1 text and the final training loss. Bottom: blinded judge panels between the two sets at equal steps, the same 120 prompts and judges as Table 6. The second judge's rulings on the two BPE panels come from a re-run with a new key after the first was revoked mid-panel.

A27

Alphabet	Distance (chars)	v2 data, 2,000 steps (released v2)	v2 data, 800 steps	v3 data, 800 steps (released v3)	v3 data, 2,000 steps	Untuned base
bytes	0	0 (12.4)	9 (11.2)	27 (11.4)	21 (13.7)	30 (10.2)
bytes	64	11 (18.5)	19 (14.6)	20 (13.9)	13 (15.3)	17 (11.1)
bytes	128	5 (14.8)	14 (13.3)	28 (14.3)	24 (15.8)	5 (9.7)
bytes	256	0 (14.0)	1 (12.4)	4 (12.6)	1 (17.0)	0 (8.7)
bytes	384	0 (13.8)	5 (13.4)	2 (13.5)	2 (15.5)	0 (8.8)
bytes	600	0 (-2.7)	0 (-3.3)	0 (0.2)	0 (0.2)	0 (1.2)
bytes	1,024	0 (2.6)	0 (4.1)	0 (2.5)	0 (1.9)	0 (1.3)
BPE	0	24 (10.2)	30 (10.6)	30 (12.0)	24 (10.2)	27 (10.8)
BPE	64	27 (13.1)	29 (12.5)	29 (13.1)	28 (13.5)	17 (10.5)
BPE	128	28 (12.1)	28 (11.6)	28 (12.0)	28 (12.4)	16 (9.2)
BPE	256	28 (12.6)	28 (12.2)	28 (12.8)	26 (12.6)	10 (8.9)
BPE	384	20 (12.0)	22 (11.6)	20 (12.0)	17 (11.6)	6 (8.3)
BPE	600	24 (11.8)	23 (11.3)	27 (12.5)	22 (11.5)	4 (7.5)
BPE	1,024	9 (8.2)	10 (8.7)	10 (10.2)	7 (8.3)	0 (5.4)

Table A27: planted-fact recall (of 30 trials, gain in nats in parentheses) for the tuning data and step count crossed: the released v2 (2,000 steps) and v3 (800 steps) models beside the two retunes of Appendix A26 (v2 data for 800 steps, v3 data for 2,000), with the untuned base. Ten recorded words in three filler texts with no window. For bytes, recall follows the documents. At 128 characters, v3 data gives 24 to 28 of 30 at either step count and the v2 data 5 to 14. The advantage holds across all three fillers (800 steps: 9, 10, and 9 of 10 versus 6, 8, and 0. 2,000 steps: 8, 9, and 7 versus 0, 5, and 0). For BPE the four tunes stay within 6 counts of one another at every distance, well above the base from 64 characters onward (at distance 0 the base can already recall 27 of 30).